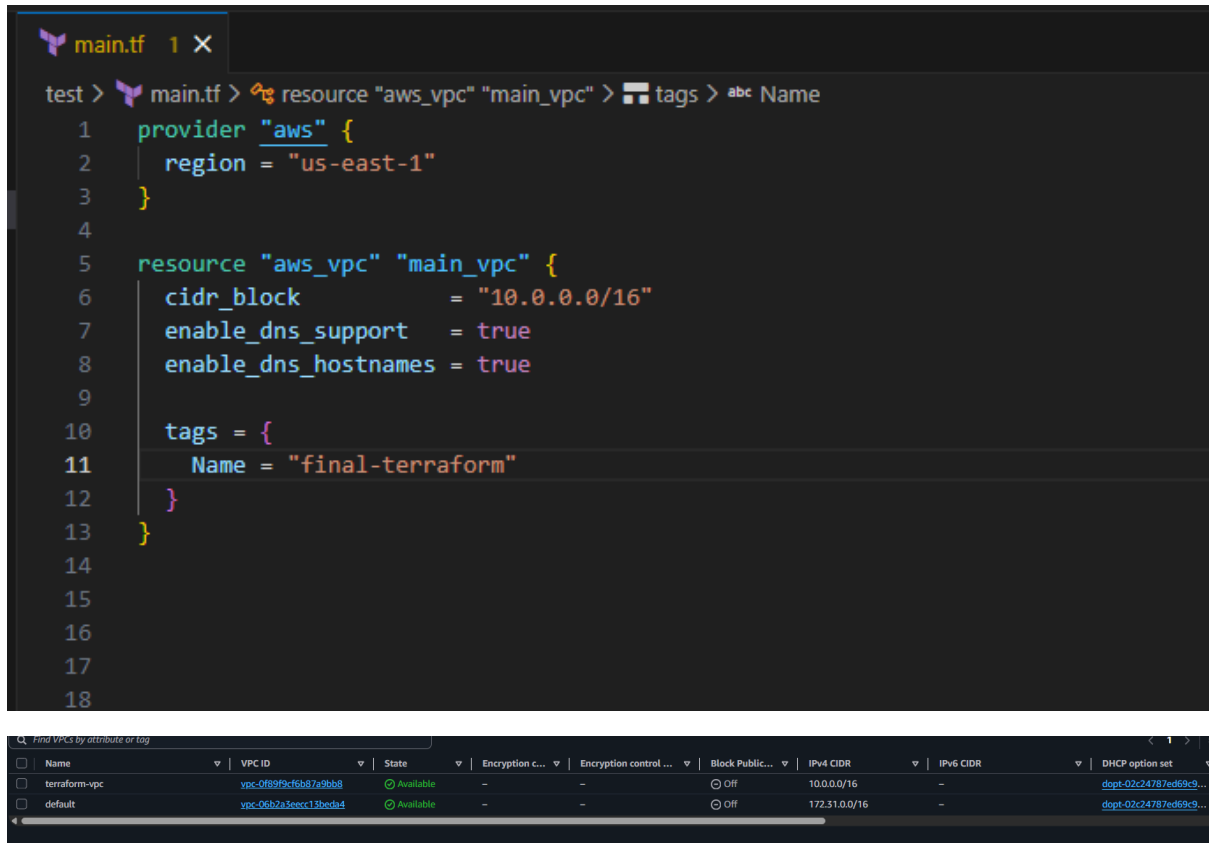


Terraform_Final_task

01: Create VPC.

Ans: creating the vpc through the terraform code added the code in main.tf



```
main.tf 1 X
test > main.tf > resource "aws_vpc" "main_vpc" > tags > abc Name
1  provider "aws" {
2      region = "us-east-1"
3  }
4
5  resource "aws_vpc" "main_vpc" {
6      cidr_block      = "10.0.0.0/16"
7      enable_dns_support = true
8      enable_dns_hostnames = true
9
10     tags = {
11         Name = "final-terraform"
12     }
13 }
14
15
16
17
18
```

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR	DHCP option set
terraform-vpc	vpc-0f89f9cf6b87a9bb8	Available	-	-	Off	10.0.0.0/16	-	dopt-02c24787ed69c9...
default	vpc-06b2a3eccc13bedc4	Available	-	-	Off	172.31.0.0/16	-	dopt-02c24787ed69c9...

2: Create Internet gateway.

Ans: creating route tables for subnets added the code in main.tf

```
main.tf x
test > main.tf > resource "aws_internet_gateway" "igw"
1 resource "aws_internet_gateway" "igw" {
2   vpc_id = aws_vpc.main_vpc.id
3
4   tags = {
5     Name = "final-igw"
6   }
7 }
```

Internet gateways (2) Info

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	-	igw-01ad41136de578db8	Attached	vpc-06b2a3eccc13beda4 default	157425729331
☐	terraform-igw	igw-081bf24c8c02e7de1	Attached	vpc-0f89f9cf6b87a9bb8 terraform-vpc	157425729331

3: Create Custom Route Table.

Ans: creating route table for subnets added the code in main.tf

```
main.tf x
test > main.tf > resource "aws_route_table" "public_rt"
1 resource "aws_route_table" "public_rt" {
2   vpc_id = aws_vpc.main_vpc.id
3
4   route {
5     cidr_block = "0.0.0.0/0"
6     gateway_id = aws_internet_gateway.igw.id
7   }
8
9   tags = {
10    Name = "public-rt"
11  }
12 }
```

Route tables (4) [Info](#)

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Own
☐	my-public-rt	rtb-07a7238e4bb98fefe	subnet-0c1a06c0ebffc99...	–	No	vpc-0f89f9cf6b87a9bb8 terraf...	157
☐	–	rtb-04f90fcf821e91e48	2 subnets	–	Yes	vpc-06b2a3eccc13beda4 default	157
☐	–	rtb-08cda563337e56c9c	–	–	Yes	vpc-0f89f9cf6b87a9bb8 terraf...	157
☐	private-route	rtb-0377c80b2c6df9afe	–	–	No	vpc-06b2a3eccc13beda4 default	157

4: Create Subnet.

Ans : creating subnets for vpc

```
main.tf x
test > main.tf > resource "aws_subnet" "public_subnet"
1 resource "aws_subnet" "public_subnet" {
2   vpc_id           = aws_vpc.main_vpc.id
3   cidr_block       = "10.0.1.0/24"
4   map_public_ip_on_launch = false
5   availability_zone = "us-east-1a"
6
7   tags = {
8     Name = "public-subnet"
9   }
10 }
```

☐	subnet-08117711232620682	vpc-06b2a3eccc13beda4 default	On	172.31.16.0/20	–
☐	subnet-08379092ab68411f2	vpc-06b2a3eccc13beda4 default	Off	172.31.48.0/20	–
☐	my-public-subnet	vpc-0f89f9cf6b87a9bb8 terraf...	Off	10.0.1.0/24	–

5: Associate subnet with Route Table.

Ans: adding the code to associate the subnet with route table

```
main.tf x
test > main.tf > resource "aws_route_table_association" "rt_assoc"
1 resource "aws_route_table_association" "rt_assoc" {
2     subnet_id      = aws_subnet.public_subnet.id
3     route_table_id = aws_route_table.public_rt.id
4 }
```

rtb-07a7238e4bb98fefe / my-public-rt

Details Routes Subnet associations Edge associations Route propagation Tags

Explicit subnet associations (1) [Edit subnet associations](#)

Q Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
my-public-subnet	subnet-0c1a06c0ebffc9952	10.0.1.0/24	-

Subnets without explicit associations (0) [Edit subnet associations](#)

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Q Find subnet association

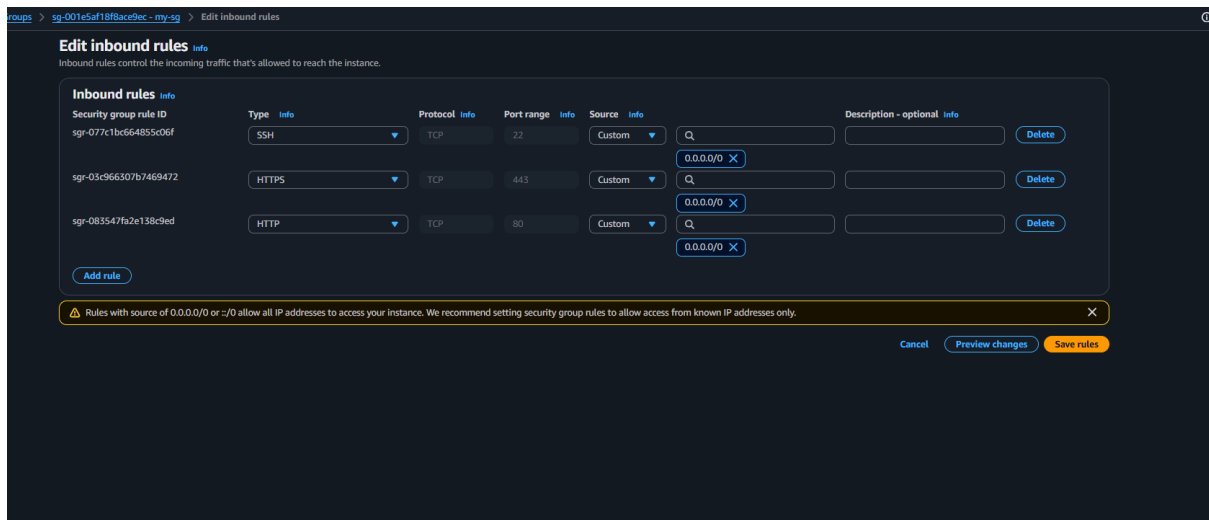
Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

No subnets without explicit associations
All your subnets are associated with a route table.

6: Create Security Group to allow port 22,80,443.

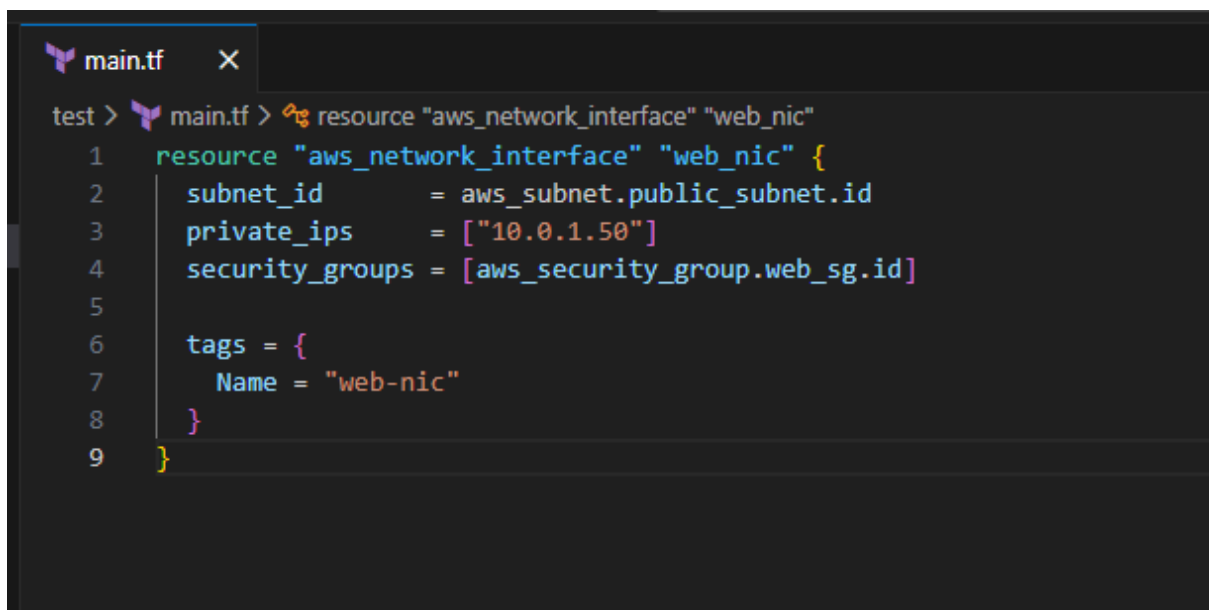
Ans: to allow the given port no added in the code

```
main.tf x
test > main.tf > resource "aws_security_group" "web_sg"
1 resource "aws_security_group" "web_sg" {
2     name      = "web-sg"
3     description = "Allow SSH, HTTP, HTTPS"
4     vpc_id    = aws_vpc.main_vpc.id
5
6     ingress {
7         description = "SSH"
8         from_port   = 22
9         to_port     = 22
10        protocol    = "tcp"
11        cidr_blocks = ["0.0.0.0/0"]
12    }
13
14    ingress {
15        description = "HTTP"
16        from_port   = 80
17        to_port     = 80
18        protocol    = "tcp"
19        cidr_blocks = ["0.0.0.0/0"]
20    }
21
22    ingress {
23        description = "HTTPS"
24        from_port   = 443
25        to_port     = 443
26        protocol    = "tcp"
27        cidr_blocks = ["0.0.0.0/0"]
28    }
29
30    egress {
31        from_port   = 0
32        to_port     = 0
33        protocol    = "-1"
34        cidr_blocks = ["0.0.0.0/0"]
35    }
36 }
```



7: Create a network interface with an ip in the subnet that was created in step 4.

Ans: upload the code for network interface



8: Assign an elastic IP to the network interface created in step 7.

Ans: added the code to assign the elastic ip to network interface which is created in step 7

```
main.tf x
test > main.tf > resource "aws_eip" "web_eip"
1 resource "aws_eip" "web_eip" {
2     domain                = "vpc"
3     network_interface      = aws_network_interface.web_nic.id
4     associate_with_private_ip = "10.0.1.50"
5
6     depends_on = [aws_internet_gateway.igw]
7
8     tags = {
9         Name = "web-eip"
10    }
11 }
```

Elastic IP addresses (1/1) info

Find elastic IP addresses by attribute or tag

Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Associated instance ID	Private IP address	Associated network interface ID
my-eip	100.29.253.192	Public IP	eipalloc-0eeb6103584ea6d5b	-	i-0a852deb7bd703608	10.0.1.50	eipassoc-08d25d2d2f990e766

100.29.253.192

View IP address usage and recommendations to release unused IPs with [Public IP insights](#)

Summary Tags

Summary	
Allocated IPv4 address 100.29.253.192	Type Public IP
Association ID eipassoc-08d25d2d2f990e766	Scope VPC
Network interface ID eni-0a59322f54d96c9d3	Network interface owner account ID 157425729331
Address pool Amazon	Network border group us-east-1
Allocation ID eipalloc-0eeb6103584ea6d5b	Reverse DNS record -
Associated instance ID i-0a852deb7bd703608	Private IP address 10.0.1.50
Public DNS ec2-100-29-253-192.compute-1.amazonaws.com	NAT Gateway ID -
Service managed -	

9: Create Ubuntu server and install/enable apache2.

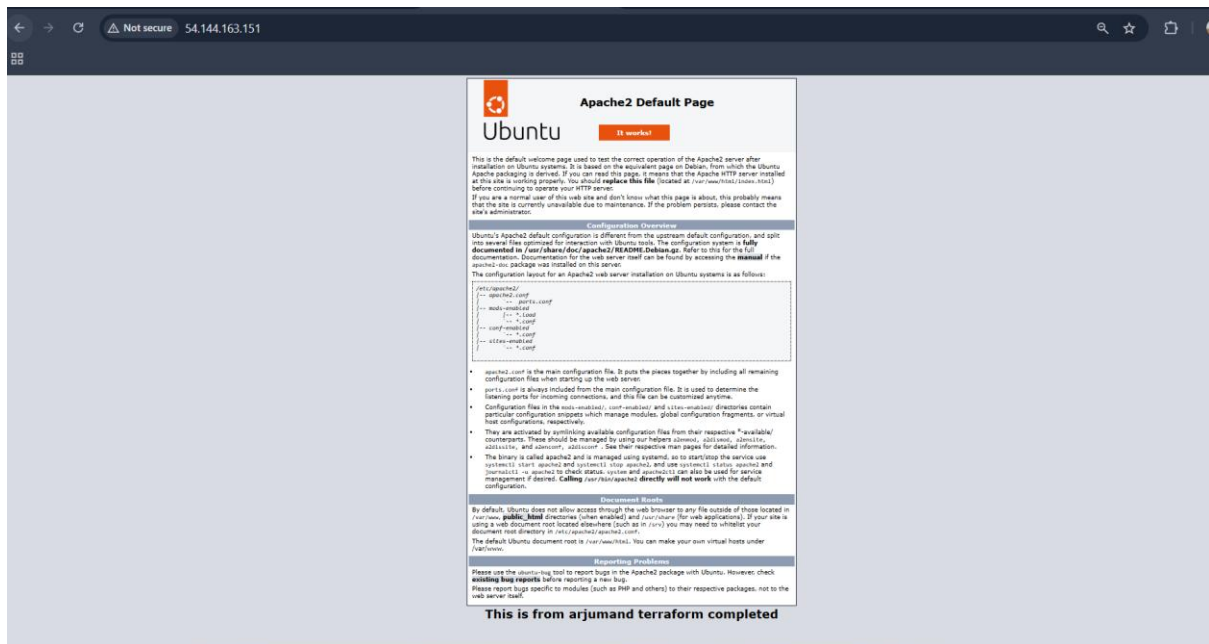
Ans: Adding the code to create ubuntu server with apache2 install/enable

```
main.tf 1 X
test > main.tf > resource "aws_instance" "web_server"
1 resource "aws_instance" "web_server" {
2     ami           = "ami-053b0d53c279acc90"
3     instance_type = "t3.micro"
4
5     network_interface {
6         network_interface_id = aws_network_interface.web_nic.id
7         device_index         = 0
8     }
9
10    user_data = <<-EOF
11        #!/bin/bash
12        apt update -y
13        apt install apache2 -y
14        systemctl enable apache2
15        systemctl start apache2
16        echo "<h1>Apache Server from Terraform 🚀</h1>" > /var/www/html/index.html
17        EOF
18
19    tags = {
20        Name = "apache-server"
21    }
22 }
```

☐	ubuntu-apach...	i-0a852deb7bd703608	 Running	 	t3.micro	 3/3 checks passed	View alarms +	us-east-1a	ec2-100-29-253-192.co...	100.29.253.192	100.29.253.192	-
☐	cb-ec2	i-09e5e02899d78af10	 Terminated	 	t3.micro	-	View alarms +	us-east-1f	-	-	-	-

```
root@ip-10-0-1-50:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-01-26 11:01:58 UTC; 14min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 1923 (apache2)
    Tasks: 55 (limit: 1008)
  Memory: 5.8M (peak: 6.3M)
     CPU: 89ms
  CGroup: /system.slice/apache2.service
          └─1923 /usr/sbin/apache2 -k start
            └─1925 /usr/sbin/apache2 -k start
              └─1927 /usr/sbin/apache2 -k start

Jan 26 11:01:58 ip-10-0-1-50 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 26 11:01:58 ip-10-0-1-50 systemd[1]: Started apache2.service - The Apache HTTP Server.
root@ip-10-0-1-50:~#
```



Below is the complete code which is been used for this task :

=====

```
provider "aws" {
  region = "us-east-1"
}
```

1. Create VPC

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
```



```
tags = {  
  Name = "my-vpc"  
}  
}
```

2. Create Internet Gateway

```
resource "aws_internet_gateway" "gw" {  
  vpc_id = aws_vpc.main.id
```

```
tags = {  
  Name = "my-igw"  
}  
}
```

3. Create Custom Route Table

```
resource "aws_route_table" "public_rt" {  
  vpc_id = aws_vpc.main.id  
  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.gw.id
```

```
}
```

```
tags = {  
  Name = "my-public-rt"  
}  
}
```

4. Create Subnet

```
resource "aws_subnet" "public_subnet" {  
  vpc_id      = aws_vpc.main.id  
  cidr_block   = "10.0.1.0/24"  
  availability_zone = "us-east-1a"  
  map_public_ip_on_launch = false
```

```
tags = {  
  Name = "my-public-subnet"  
}  
}
```

5. Associate Subnet with Route Table

```
resource "aws_route_table_association" "rta" {  
  subnet_id = aws_subnet.public_subnet.id
```

```
route_table_id = aws_route_table.public_rt.id
}
```

6. Create Security Group (22, 80, 443)

```
resource "aws_security_group" "web_sg" {
  name      = "my-sg"
  description = "Allow SSH, HTTP, HTTPS"
  vpc_id    = aws_vpc.main.id
```

```
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
ingress {  
    from_port = 443  
    to_port   = 443  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
tags = {  
    Name = "my-sg"  
}  
}
```

7. Create Network Interface

```
resource "aws_network_interface" "my_eni" {
```

```
subnet_id    = aws_subnet.public_subnet.id
private_ips  = ["10.0.1.50"]
security_groups = [aws_security_group.web_sg.id]

tags = {
    Name = "my-eni"
}

# 8. Assign Elastic IP to Network Interface
resource "aws_eip" "my_eip" {
    domain      = "vpc"
    network_interface = aws_network_interface.my_eni.id
    associate_with_private_ip = "10.0.1.50"

    depends_on = [aws_internet_gateway.gw]

    tags = {
        Name = "my-eip"
    }
}
```

9. Create Ubuntu Server and Install Apache

```
resource "aws_instance" "web_server" {  
  ami          = "ami-0b6c6ebed2801a5cb" # Ubuntu 22.04  
  LTS (us-east-1)  
  instance_type = "t3.micro"  
  
  network_interface {  
    network_interface_id =  
aws_network_interface.my_eni.id  
    device_index        = 0  
  }  
  
  user_data = <<-EOF  
    #!/bin/bash  
    apt update -y  
    apt install apache2 -y  
    systemctl start apache2  
    systemctl enable apache2  
    EOF  
  
  tags = {  
    Name = "ubuntu-apache-server"
```

}

}